

EE3621: Digital Signal Processing

Lecture 27: DSP for Image Processing — 2D DFT, Filtering & Transforms (III B.Tech EEE)

Lecture Notes (40-Minute Plan)

1 Lecture Plan (40 Minutes Breakdown)

- **00:00 – 05:00 (5 mins):** Welcome & 2D Signals. Introduction to digital images, spatial sampling, and aliasing.
 - **05:00 – 12:00 (7 mins):** 2D DTFT and 2D DFT. Spatial frequency spectrum, separability, and row-column FFT computations.
 - **12:00 – 18:00 (6 mins):** 2D Convolution & Filtering. Derivations of separability and computational savings.
 - **18:00 – 24:00 (6 mins):** Spatial Frequency Domain Filtering. Low-pass vs. High-pass, Gaussian, and Laplacian.
 - **24:00 – 30:00 (6 mins):** 2D Discrete Cosine Transform (DCT). Energy compaction and JPEG block-level processing.
 - **30:00 – 36:00 (6 mins):** Edge Detection. Gradient operators (Sobel), gradient magnitude/direction, and Canny steps.
 - **36:00 – 40:00 (4 mins):** Summary, Key Formulas, and Checkpoint Questions.
-

2 2D Signals: The Digital Image

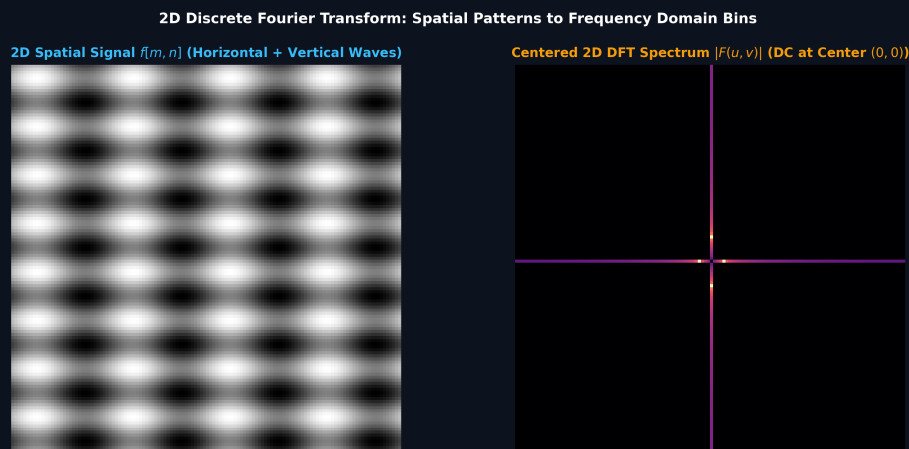


Figure 1: 2D Discrete Fourier Transform: Mapping Spatial Image Brightness Patterns to Frequency Coordinates.

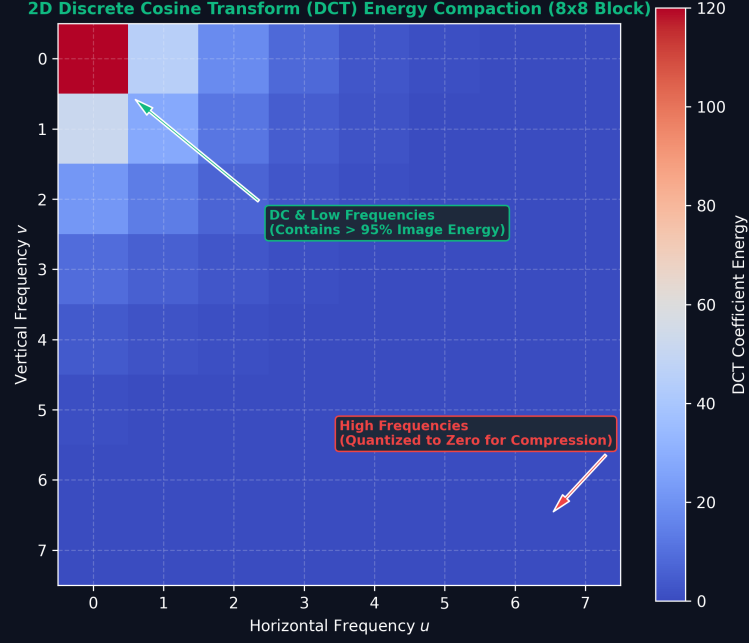


Figure 2: 2D Discrete Cosine Transform (DCT) Energy Compaction Matrix in 8×8 JPEG Compression.

In traditional 1D digital signal processing, a signal $x[n]$ is typically a sequence of values indexed by time (e.g., an audio recording). In the realm of image processing, however, a digital image is fundamentally a **2D discrete signal**, denoted by $f[m, n]$. Here, m and n are discrete spatial coordinates representing the rows and columns of a pixel grid.

2.1 Pixel Values and Domains

When we sample an optical image, we discretize both the spatial coordinates and the amplitude (intensity) of the light.

- **Grayscale Images:** The function $f[m, n]$ represents a single intensity value for each coordinate pair. This is typically mapped on an 8-bit scale, ranging from 0 (pure black) to 255 (pure white). Intermediate values represent varying shades of gray.
- **Color Images (RGB):** A color image is a multidimensional signal. Each pixel consists of three separate channels: Red, Green, and Blue. Thus, the image can be viewed as three stacked 2D signals: $f_R[m, n]$, $f_G[m, n]$, and $f_B[m, n]$, or as a single 3D tensor $f[m, n, c]$ where $c \in \{0, 1, 2\}$.

2.2 Spatial Sampling and Aliasing

Just as a 1D continuous audio signal must be sampled above the Nyquist rate to avoid temporal aliasing (where high frequencies masquerade as low frequencies), a continuous 2D optical image $f_c(x, y)$ must be sampled with an adequately fine spatial resolution. The spatial resolution is determined by the density of the sensor pixels.

Physical Intuition: Imagine taking a digital photograph of a brick wall from far away, or a person wearing a shirt with very fine, densely packed stripes. These physical objects represent **high spatial frequencies** because the visual intensity changes very rapidly over a small spatial distance.

If your camera's sensor does not have enough pixels to capture these rapid changes (i.e., you are sampling below the spatial Nyquist rate), the high-frequency stripes will fold over

into lower frequencies. Visually, this manifests as **Moiré patterns**—strange, wavy, artificial low-frequency artifacts that do not exist in the real object.

This is **spatial aliasing**, which is conceptually identical to the 1D aliasing demonstrated below (from our earlier analysis on frequency folding). Notice how high-frequency spectral components wrap around and corrupt the primary band.

3 2D DTFT and 2D DFT

To analyze the frequency content of images—to understand where the rapid changes (edges) and slow changes (smooth areas) lie—we must extend our Fourier analysis tools into two dimensions.

3.1 The 2D Discrete-Time Fourier Transform (DTFT)

The 2D DTFT is the theoretical bridge that converts a discrete spatial domain signal $f[m, n]$ of infinite extent into a continuous spatial frequency domain representation, denoted as $F(e^{j\omega_x}, e^{j\omega_y})$:

$$F(e^{j\omega_x}, e^{j\omega_y}) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} f[m, n] e^{-j(\omega_x m + \omega_y n)} \quad (1)$$

Physical Intuition in the Frequency Domain:

- ω_x (**Horizontal Frequency**): Represents the rate of variation across the horizontal axis (columns). A high ω_x means rapid changes horizontally, which corresponds to vertical edges.
- ω_y (**Vertical Frequency**): Represents the rate of variation down the vertical axis (rows). A high ω_y corresponds to horizontal edges.
- **The 2D Magnitude Spectrum**: When we plot $|F(e^{j\omega_x}, e^{j\omega_y})|$ as an image, **low frequencies** are concentrated at the center ($\omega_x = 0, \omega_y = 0$). These represent smooth shading, flat regions, and the overall average brightness (DC component). **High frequencies** extend out to the periphery, representing sharp edges, fine textures, and random noise.

3.2 The 2D Discrete Fourier Transform (DFT)

In practice, images are not infinite; they have a finite size of $M \times N$ pixels. Therefore, we use the 2D DFT, which samples the continuous DTFT at discrete intervals.

The forward 2D DFT is defined as:

$$F[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] W_M^{km} W_N^{ln} \quad (2)$$

where the standard twiddle factors are $W_M = e^{-j2\pi/M}$ and $W_N = e^{-j2\pi/N}$.

The Inverse 2D DFT (IDFT) brings us back to the spatial domain:

$$f[m, n] = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} F[k, l] W_M^{-km} W_N^{-ln} \quad (3)$$

3.3 Proof of Separability and Row-Column Computation

A remarkable and computationally vital property of the 2D DFT is its **separability**.

Theorem: The 2D DFT can be computed by performing 1D DFTs along the rows, followed by 1D DFTs along the columns.

Step-by-Step Derivation:

$$F[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] e^{-j\frac{2\pi}{M}km} e^{-j\frac{2\pi}{N}ln} \quad (4)$$

$$F[k, l] = \sum_{m=0}^{M-1} e^{-j\frac{2\pi}{M}km} \left[\sum_{n=0}^{N-1} f[m, n] e^{-j\frac{2\pi}{N}ln} \right] \quad (5)$$

Notice that the expression in the brackets is exactly a 1D DFT applied to the m -th row of the image. Let us call this intermediate result $G[m, l]$:

$$G[m, l] = \sum_{n=0}^{N-1} f[m, n] W_N^{ln} \quad (6)$$

Now, substitute $G[m, l]$ back into the outer summation:

$$F[k, l] = \sum_{m=0}^{M-1} G[m, l] W_M^{km} \quad (7)$$

This final expression is simply a 1D DFT applied to the l -th column of the intermediate array G .

In compact matrix notation, if $\mathbf{f}$ is the image matrix, this separability is elegantly written as:

$$\mathbf{F} = \mathbf{W}_M \mathbf{f} \mathbf{W}_N \quad (8)$$

Computational Savings: Why is separability so important? It drastically reduces the number of mathematical operations. A naive approach to computing the 2D DFT directly via the double summation requires $\mathcal{O}(M^2 N^2)$ operations. For a 1000×1000 image, this is 10^{12} operations.

Using the Row-Column approach with 1D Fast Fourier Transforms (FFTs), we compute M row-wise FFTs of length N , followed by N column-wise FFTs of length M . The total complexity drops to $\mathcal{O}(MN \log_2(MN))$. For a 1000×1000 image, this is roughly 2×10^7 operations—a speedup of $50,000\times$!

4 2D Convolution

In image processing, linear shift-invariant (LSI) filtering is performed using 2D convolution. If we have a 2D filter $h[m, n]$ (often called a kernel or mask), the filtered output image $g[m, n]$ is given by:

$$g[m, n] = h[m, n] * * f[m, n] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} h[k, l] f[m - k, n - l] \quad (9)$$

The process involves flipping the kernel h horizontally and vertically, sliding it over the image f , and computing the sum of element-wise multiplications at every position.

4.1 Separable 2D Filters

Just like the DFT, certain filters possess the property of separability. A 2D filter is **separable** if it can be expressed as the outer product of a 1D vertical filter $h_x[m]$ and a 1D horizontal filter $h_y[n]$:

$$h[m, n] = h_x[m] h_y[n] \quad (10)$$

Theorem: 2D convolution with a separable filter is mathematically equivalent to two sequential 1D convolutions.

Step-by-Step Proof:

$$g[m, n] = \sum_k \sum_l (h_x[k] h_y[l]) f[m - k, n - l] \quad (11)$$

$$g[m, n] = \sum_k h_x[k] \left(\sum_l h_y[l] f[m - k, n - l] \right) \quad (12)$$

Define the inner summation as an intermediate image $w[m - k, n]$, which represents the image rows convolved with h_y :

$$w[m - k, n] = \sum_l h_y[l] f[m - k, n - l] \quad (13)$$

Substitute w back into the outer sum:

$$g[m, n] = \sum_k h_x[k] w[m - k, n] \quad (14)$$

This is a 1D convolution of the intermediate columns with h_x .

Computational Savings: Suppose we are applying a $K \times K$ filter to an $M \times N$ image. Standard 2D convolution requires $K^2 \times M \times N$ multiplications. A separable convolution requires $2K \times M \times N$ multiplications. For a 15×15 Gaussian blur, the computational load is reduced by 86.6%, allowing real-time video processing.

5 Spatial Frequency Domain Filtering

Filters applied in the spatial domain using convolution have direct counterparts in the frequency domain, where convolution becomes simple element-wise multiplication:

$$G[k, l] = H[k, l] \cdot F[k, l] \quad (15)$$

By manipulating the 2D spectrum, we can perform various enhancement operations:

- **Low-pass Filtering:** Attenuates high frequencies. Used for noise reduction, skin smoothing, and blurring.
- **High-pass Filtering:** Attenuates low frequencies. Used for edge detection, image sharpening, and feature extraction.
- **Bandpass Filtering:** Allows a specific ring of frequencies to pass. Used to isolate specific repetitive textures or remove periodic noise.

An **ideal** circular 2D low-pass filter abruptly cuts off all frequencies above a certain radius cutoff. However, in the spatial domain, the IDFT of an ideal cylinder is a 2D sinc function (a "sombbrero" function). This function has infinite undulating tails, causing heavy **ringing artifacts** (Gibbs phenomenon) around sharp edges. To avoid ringing, we use practical filters with smooth, gradual roll-offs.

6 Gaussian and Laplacian Filters

6.1 Gaussian Smoothing (Low-pass)

The 2D Gaussian filter is ubiquitous in computer vision. It is the only filter whose Fourier transform is also a Gaussian, meaning it possesses optimal localization in both the spatial and frequency domains.

$$G_\sigma[m, n] = \frac{1}{2\pi\sigma^2} e^{-\frac{m^2+n^2}{2\sigma^2}} \quad (16)$$

The standard deviation σ controls the amount of blur. It is completely separable since $e^{-(m^2+n^2)} = e^{-m^2} e^{-n^2}$.

6.2 Laplacian Operator (High-pass)

The Laplacian is an isotropic (rotation-invariant) 2D second-order spatial derivative.

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad (17)$$

Because second derivatives are extremely sensitive to high-frequency noise, the Laplacian is almost never used on raw images. Instead, the image is first smoothed with a Gaussian, creating the **Laplacian of Gaussian (LoG)** filter: $L = \nabla^2 G_\sigma$. The LoG filter resembles a Mexican hat and is used for blob detection.

7 2D DCT and JPEG Compression

The **Discrete Cosine Transform (DCT)** uses only real cosine basis functions and implies symmetric boundary conditions, avoiding edge artifacts associated with the DFT. The 2D Type-II DCT for an $M \times N$ image block is:

$$F[k, l] = \frac{4c[k]c[l]}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] \cos \left[\frac{(2m+1)k\pi}{2M} \right] \cos \left[\frac{(2n+1)l\pi}{2N} \right] \quad (18)$$

Energy Compaction: The DCT concentrates almost all of the visual signal's energy into a very small number of low-frequency coefficients located in the top-left corner of the $F[k, l]$ matrix.

JPEG Pipeline: 1. **Color Space Conversion and Subsampling:** Convert to YCbCr and downsample.

2. **Blocking:** Slice image into 8×8 non-overlapping blocks.

3. **2D DCT:** Transform each spatial block to a frequency matrix.

4. **Quantization (The Lossy Step):** Divide by a Quantization Table and round to nearest integer, crushing high frequencies to zero.

5. **Zigzag Scanning:** Group zeros together.

6. **Entropy Coding:** Compress using Run-Length Encoding (RLE) and Huffman coding.

8 Edge Detection & Gradient Operators

Mathematically, an edge corresponds to a local maximum of the first spatial derivative of the image intensity.

$$\nabla f = \begin{bmatrix} G_x \\ G_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} \quad (19)$$

- **Gradient Magnitude:** $|\nabla f| = \sqrt{G_x^2 + G_y^2}$
- **Gradient Direction:** $\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$

8.1 Sobel Operators

We approximate derivatives using finite differences. The Sobel operators apply derivative and smoothing simultaneously.

$$H_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad H_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (20)$$

Worked Numerical Example: Consider a 3×3 image patch with a vertical edge:

$$f = \begin{bmatrix} 50 & 50 & 200 \\ 50 & 50 & 200 \\ 50 & 50 & 200 \end{bmatrix}$$

$$G_x = (-1 \cdot 50) + (1 \cdot 200) + (-2 \cdot 50) + (2 \cdot 200) + (-1 \cdot 50) + (1 \cdot 200) = 600$$

$$G_y = (-1 \cdot 50 - 2 \cdot 50 - 1 \cdot 200) + (1 \cdot 50 + 2 \cdot 50 + 1 \cdot 200) = 0$$

$$|\nabla f| = \sqrt{600^2 + 0^2} = 600 \quad (\text{Massive vertical edge})$$

8.2 The Canny Edge Detector

The Canny Edge Detector is a multi-stage intelligent algorithm: 1. **Gaussian Smoothing:** Remove high-frequency noise. 2. **Gradient Calculation:** Compute Sobel gradients to find magnitude and angle. 3. **Non-Maximum Suppression (NMS):** Thin edges to a single pixel wide. 4. **Hysteresis Thresholding:** Use dual thresholds (T_{low} and T_{high}). Connect weak edges to strong edges to maintain continuous boundaries.

9 Key Formulas Summary

Concept	Mathematical Formulation
2D DTFT	$F(e^{j\omega_x}, e^{j\omega_y}) = \sum_m \sum_n f[m, n] e^{-j(\omega_x m + \omega_y n)}$
2D DFT	$F[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] W_M^{km} W_N^{ln}$
Separable 2D DFT	$\mathbf{F} = \mathbf{W}_M \mathbf{f} \mathbf{W}_N$
2D Convolution	$g[m, n] = \sum_k \sum_l h[k, l] f[m - k, n - l]$
Gaussian Filter	$G_\sigma[m, n] = \frac{1}{2\pi\sigma^2} e^{-(m^2+n^2)/(2\sigma^2)}$
Laplacian	$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$
Gradient Magnitude	$ \nabla f = \sqrt{G_x^2 + G_y^2}$
Gradient Direction	$\theta = \tan^{-1}(G_y/G_x)$

10 Checkpoint Questions

Q1: A 2D spatial filter has the following 3×3 kernel: $h = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$. Prove mathematically that this filter is separable and explicitly identify its 1D vertical and horizontal components.

Answer: A 2D filter is separable if it can be written as an outer product of a column vector

h_x and a row vector h_y . Let $h_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$ and $h_y = [1 \ 2 \ 1]$. Multiplying them out gives:

$h_x h_y = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$. Since the 2D matrix decomposes perfectly, it is proven to be completely separable.

Q2: During the JPEG compression pipeline, an 8×8 Discrete Cosine Transform is computed. Why does the JPEG algorithm perform a complicated “Zigzag scanning” pattern immediately after the quantization step, instead of just reading the

matrix row by row?

Answer: The quantization step heavily suppresses high-frequency coefficients, replacing them with literal zeros. By reading diagonally (zigzag) starting from the top-left (low frequencies) and moving towards the bottom-right (high frequencies), all the large non-zero coefficients are neatly clustered at the beginning. This leaves a continuous, unbroken run of zeros at the end, allowing Run-Length Encoding (RLE) to drastically compress the string and minimize the file size.

Q3: Suppose a Canny Edge Detector identifies a long, continuous edge (e.g., a wire). However, due to lighting, the middle portion of the wire has a gradient magnitude slightly below the upper threshold T_{high} . How does the algorithm prevent the wire's edge from breaking in half?

Answer: The algorithm uses **Hysteresis Thresholding** with two thresholds (T_{low} and T_{high}). While the strong ends of the wire are kept because they exceed T_{high} , the dim middle section falls between T_{low} and T_{high} . Because this “weak” section physically touches (connects to) the “strong” edge pixels at either end, the hysteresis logic promotes the weak pixels to strong status. Consequently, the edge survives thresholding and remains entirely contiguous.

11 Extended Topics: Mathematical Properties of the 2D DFT

11.1 Translation (Spatial Shift)

Shifting an image corresponds to a phase shift in the frequency domain.

$$f[m - m_0, n - n_0] \longleftrightarrow F[k, l] e^{-j \frac{2\pi}{M} k m_0} e^{-j \frac{2\pi}{N} l n_0} \quad (21)$$

The magnitude spectrum $|F[k, l]|$ remains identical; only the phase changes.

11.2 Rotation and Scaling

If an image is rotated by θ , its spectrum is also rotated by θ . Scaling an image causes inverse scaling in the frequency domain:

$$f(a \cdot x, b \cdot y) \longleftrightarrow \frac{1}{|ab|} F\left(\frac{u}{a}, \frac{v}{b}\right) \quad (22)$$

Tiny, sharp details (narrow spatial width) correspond to very high frequencies.

12 Extended Topics: Advanced Edge Detection Operators

12.1 The Prewitt Operator

Provides a standard average rather than a weighted average.

$$H_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}, \quad H_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad (23)$$

12.2 The Scharr Operator

Provides better rotational symmetry than Sobel.

$$H_x = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix} \quad (24)$$

13 Deep Dive: Zero-Padding and Linear Convolution

To perform true **Linear Convolution** using the DFT, we must zero-pad both the image and the filter. If the image is $M \times N$ and the filter is $K \times L$, we must pad both to:

$$P \geq M + K - 1 \tag{25}$$

$$Q \geq N + L - 1 \tag{26}$$

Only then can we multiply the DFTs and avoid circular convolution artifacts.